

**The Ethics of AI-Assisted Code Migration in Regulated Financial Systems: Accountability,
Transparency, and Human Oversight in LLM-Driven Software Conversion**

Ronit Sharma

Independent Researcher

February 2026

Abstract

Large language models (LLMs) such as GitHub Copilot are increasingly deployed not merely to assist developers with code completion, but to perform large-scale automated code migration—converting entire legacy codebases from one language and framework to another. In regulated financial services, where software systems manage sensitive instruments such as employee stock option plans, loan servicing platforms, and trading infrastructure, this practice introduces a distinct and underexplored set of ethical challenges at the intersection of AI ethics, software engineering, and financial regulation. This paper examines the ethical dimensions of AI-assisted code migration within enterprise financial systems, drawing on firsthand practitioner experience deploying LLM-driven conversion workflows at a major financial institution. Three core ethical tensions are identified: an accountability gap when AI-generated code fails in production, a transparency deficit that impedes regulatory audit and compliance verification, and an automation bias that leads teams to place unwarranted trust in AI-converted code. A practitioner-grounded ethical framework is proposed, consisting of mandatory human review checkpoints, provenance documentation for AI-generated components, and differential testing protocols that verify functional equivalence against original systems. This work moves beyond abstract ethical principles toward actionable guidance rooted in the realities of enterprise software engineering, and calls on industry leaders, compliance teams, and AI tool providers to co-develop standardized ethical review processes for AI-generated code in regulated industries.

Keywords: AI ethics, code migration, large language models, GitHub Copilot, financial services, software engineering ethics, accountability, transparency, automation bias

1. Introduction

The adoption of artificial intelligence in software engineering has accelerated at a pace that outstrips the development of ethical frameworks to govern its use. Large language models, and specifically AI-powered coding assistants such as GitHub Copilot, Amazon CodeWhisperer, and Cursor, have moved from experimental novelties to standard tools embedded in professional development workflows across industries. A 2025 industry survey found that over 97% of software companies have adopted AI tools in their engineering processes, with code generation representing the dominant use case (Techreviewer, 2025). These tools promise dramatic productivity gains: faster development cycles, reduced manual effort, and the ability to automate tasks that previously required extensive human labor.

Among the most consequential of these tasks is code migration—the process of converting a software system from one programming language, framework, or platform to another. Legacy code migration has long been one of the most expensive and error-prone undertakings in enterprise software engineering. Financial institutions, in particular, maintain vast codebases written in older technologies that must be modernized to meet evolving security requirements, regulatory mandates, and business needs. The appeal of using LLMs to automate or accelerate this process is self-evident: what once required months of painstaking manual rewriting can now be accomplished in a fraction of the time.

However, the deployment of AI-assisted code migration in regulated industries raises ethical questions that the current discourse has not adequately addressed. The existing literature on AI ethics in software engineering tends to focus on general concerns such as algorithmic bias, intellectual property, and data privacy (Morley et al., 2020). While these are important, they do not capture the specific ethical tensions that emerge when an LLM is tasked with rewriting code

that governs financial calculations, handles sensitive employee data, or interfaces with regulatory reporting systems. The consequences of failure in these contexts are not abstract—they are measured in miscalculated compensation, compliance violations, and erosion of institutional trust.

This paper addresses that gap. Drawing on firsthand experience deploying LLM-driven code conversion workflows at a major financial institution, it identifies three core ethical tensions inherent in AI-assisted code migration and proposes a practitioner-grounded ethical framework designed to mitigate them. The analysis is deliberately generalized beyond any single language pair or platform, because the ethical challenges described here apply wherever AI is used to rewrite code in environments where accuracy, auditability, and regulatory compliance are paramount.

2. Background and Related Work

2.1 AI-Assisted Code Generation and Migration

AI-powered code generation tools have matured rapidly since the introduction of GitHub Copilot in 2021. Built on large language models trained on billions of lines of publicly available source code, these tools can suggest code completions, generate entire functions from natural-language prompts, and—increasingly—perform cross-language translation of code modules. Research by Barke, James, and Polikarpova (2023) documented how programmers interact with code-generating models, identifying patterns of both productive collaboration and uncritical acceptance of AI suggestions. Vaithilingam, Zhang, and Glassman (2022) found significant gaps between developer expectations and the actual usability of LLM-generated code, particularly around correctness verification and debugging.

Code migration represents a qualitatively different challenge from code completion. Where code completion assists a developer in writing new logic, code migration requires the AI to comprehend the semantics of an existing system, faithfully translate its behavior into a different language or framework, and produce output that compiles, executes, and yields identical results. Sauvola et al. (2024) examined the future of generative AI in software development and highlighted that accountability, reliability, and ethical governance remain critical open questions, particularly in legacy system modernization scenarios where AI-generated code must meet the functional specifications of its predecessor.

2.2 Ethics of AI in Software Engineering

The broader literature on AI ethics in software engineering has established several foundational concerns. Morley, Floridi, Kinsey, and Elhalal (2020) provided an initial review of publicly available AI ethics tools and methods, identifying transparency, accountability, and fairness as recurring themes across ethical frameworks. More recently, O’Sullivan (2024) examined the ethics of generative AI in code production specifically, arguing that traditional software engineering norms around code ownership and quality assurance require significant rethinking in the age of AI-assisted development.

Within financial services, the ethical dimensions of AI use have received attention primarily in the context of algorithmic decision-making—credit scoring, fraud detection, and trading systems (Coeckelbergh, 2022). The ethics of AI in the infrastructure layer of financial technology—the code that implements these systems—has received comparatively little scholarly attention. This paper seeks to begin filling that gap by examining the ethical implications of the tools used to build and maintain the software on which financial services depend on.

2.3 The Regulatory Context

Financial institutions operate under stringent regulatory frameworks that impose specific requirements on the software systems they deploy. Regulations such as the Sarbanes-Oxley Act in the United States, the Markets in Financial Instruments Directive (MiFID II) in Europe, and various banking supervisory guidelines mandate that institutions maintain auditable records of how their systems process financial data, execute calculations, and generate reports. These requirements presuppose that the institution can explain how its code works—a presupposition that becomes significantly more difficult to satisfy when portions of that code have been generated or rewritten by an AI system whose internal reasoning is opaque.

3. Three Ethical Tensions in AI-Assisted Code Migration

This section identifies and examines three core ethical tensions that arise when LLMs are used to migrate code in regulated financial environments. These tensions were identified through direct practitioner experience and are grounded in the operational realities of deploying AI-driven code conversion workflows in production settings.

3.1 The Accountability Gap

Traditional software engineering operates under a clear chain of accountability: the developer who writes a piece of code is responsible for its correctness. When a function miscalculates a value, introduces a security vulnerability, or fails to handle an edge case, the author of that code—and the review process that approved it—bears responsibility. This accountability chain is foundational to engineering ethics, codified in professional standards such as the ACM Code of Ethics (ACM, 2018), and enforced through institutional practices including code review, testing, and sign-off procedures.

AI-assisted code migration disrupts this chain. When an LLM converts a module from one language to another, the resulting code is authored by neither the original developer nor the engineer who initiated the migration. The developer who prompted the AI may not fully understand every line of the generated output, particularly in large-scale migrations involving thousands of lines of code across dozens of modules. Yet that developer is typically expected to take ownership of the converted code as though they had written it themselves.

This creates what may be termed an accountability gap: a structural disconnect between the assignment of responsibility and the actual capacity to exercise it. In a financial context, this gap has concrete consequences. Consider a scenario in which an AI-converted function calculates employee stock option vesting schedules. If the converted code introduces a subtle rounding error that goes undetected through standard testing, the financial impact could affect thousands of employees before the error is discovered. Who is accountable? The developer who ran the migration tool? The team lead who approved the deployment? The vendor who provided the AI model? The institution that chose to use AI for the task? The diffusion of responsibility across these actors represents a genuine ethical concern that existing frameworks do not adequately address.

3.2 The Transparency Deficit

Transparency is a cornerstone of both software engineering best practices and financial regulatory compliance. Developers are expected to write code that is readable, well-documented, and amenable to review. Regulators expect institutions to demonstrate that their systems perform as intended and to provide audit trails that trace calculations back to their source logic.

LLM-generated code presents a transparency challenge on two levels. At the generation level, the model cannot explain why it chose a particular translation strategy. Unlike a human

developer who can articulate the reasoning behind a design decision, an LLM produces output through statistical inference over its training data. When asked to convert a complex business logic module, the model may produce functionally correct code that is structurally different from the original in ways that obscure the mapping between source and target. This makes it difficult for reviewers to verify that the conversion faithfully preserves the original system's behavior.

At the audit level, the opacity of AI-generated code complicates regulatory compliance. When an auditor asks how a particular calculation is implemented, the institution must be able to trace the logic from requirement to code. If that code was generated by an AI, the provenance chain is broken. The institution cannot point to a design document or a developer's reasoning; it can only point to an AI model that processed an input and produced an output. In regulated industries where explainability is not optional but mandated, this transparency deficit represents a material compliance risk.

3.3 Automation Bias

Automation bias refers to the tendency of humans to favor suggestions from automated systems over their own judgment, even when the automated output is incorrect (Parasuraman & Manzey, 2010). In the context of AI-assisted code migration, automation bias manifests in a specific and dangerous form: teams may place unwarranted trust in AI-converted code, particularly when the output compiles successfully and passes surface-level tests.

The risk is amplified by the nature of code migration itself. Migration projects are typically undertaken under time pressure and resource constraints—the very conditions under which automation bias is most pronounced. When an AI tool converts hundreds of files in hours rather than months, the sheer volume of output creates an implicit expectation that human review will be cursory rather than thorough. Developers may scan the converted code for obvious errors

but lack the time or institutional mandate to perform the deep, line-by-line comparison against the original that would be necessary to catch subtle semantic differences.

In financial systems, the consequences of automation bias in code migration are particularly severe. Financial calculations often involve edge cases, rounding conventions, and date-handling logic that vary between languages and frameworks. An AI model may produce code that handles the common case correctly while silently diverging on edge cases that represent a small but financially significant portion of transactions. If the review process is weakened by automation bias, these divergences may persist in production for extended periods, compounding their impact.

4. A Practitioner-Grounded Ethical Framework

The ethical tensions identified above are not merely theoretical concerns; they are operational risks that require practical mitigation. This section proposes a three-part ethical framework for AI-assisted code migration in regulated industries. The framework is designed to be implementable within existing enterprise development workflows without requiring wholesale organizational restructuring.

4.1 Principle 1: Mandatory Human Review Checkpoints

The first principle requires that human review be structurally embedded in the migration workflow at defined checkpoints, rather than relegated to a final quality assurance step. Specifically, the framework mandates human review at three stages: pre-migration scoping, where a senior engineer evaluates which modules are appropriate candidates for AI-assisted migration and which require manual conversion due to complexity or criticality; mid-migration validation, where converted modules are reviewed at the individual component level before

integration; and post-migration certification, where a designated engineer formally attests that the converted system has been reviewed and tested to a standard commensurate with the risk profile of the application.

This principle directly addresses the accountability gap by establishing clear ownership at each stage of the migration process. The engineer who certifies the final output is not merely signing off on an AI's work; they are accepting personal and professional responsibility for the code's correctness in the same manner that a structural engineer certifies the integrity of a building. This framing recenters human judgment as the ultimate authority in the migration process, with AI serving as a tool that accelerates but does not replace the engineer's evaluative role.

4.2 Principle 2: Provenance Documentation

The second principle requires comprehensive provenance documentation for all AI-generated or AI-modified code. Each converted component must be accompanied by a record that specifies the source module and its language or framework version, the AI model and version used for the conversion, the date and configuration of the migration run, any manual modifications made to the AI-generated output after conversion, and the identity of the engineer who reviewed and approved the converted code.

This provenance trail serves multiple purposes. For internal engineering teams, it provides the context necessary to understand and maintain converted code over time. For compliance and audit functions, it establishes the explainability chain that regulators require. For the institution as a whole, it creates an evidentiary record that can be used to assess the reliability of AI-assisted migration as a methodology and to identify patterns of error that may warrant changes in process or tooling.

Provenance documentation also addresses the transparency deficit by making the AI's role in the codebase visible and traceable. Rather than treating AI-generated code as indistinguishable from human-written code, the framework requires that its origins be explicitly recorded and accessible to anyone who interacts with it. This transparency is not merely a bureaucratic requirement; it is an ethical obligation to ensure that those who depend on the code—whether they are end users, regulators, or future developers—can understand how it came to exist.

4.3 Principle 3: Differential Testing Protocols

The third principle mandates differential testing that goes beyond compilation and unit test passage to verify functional equivalence between the original and converted systems. Standard testing practices are insufficient for AI-assisted code migration because they typically test the converted code in isolation, confirming that it produces expected outputs for a predefined set of inputs. They do not systematically compare the behavior of the converted system against the original across the full range of inputs that the system encounters in production.

Differential testing addresses this limitation by running both the original and converted systems in parallel against a comprehensive corpus of real or representative data, then comparing their outputs at a granular level. Discrepancies are flagged for human review, with particular attention to edge cases involving date boundaries, rounding behavior, null handling, currency conversions, and other domains where subtle differences between language implementations are known to produce divergent results.

This principle directly counters automation bias by replacing subjective confidence in AI output with objective, measurable evidence of equivalence. It shifts the burden of proof from the engineer—who must otherwise trust that the AI got it right—to the testing infrastructure, which

provides empirical confirmation or refutation. In doing so, it establishes a standard of verification that is commensurate with the risk profile of the systems being migrated, and it ensures that the efficiency gains of AI-assisted migration are not purchased at the cost of reliability.

5. Discussion

The framework proposed here is deliberately conservative in its assumptions about AI capability. It does not presume that LLMs will become perfectly reliable code translators, nor does it dismiss their utility. Instead, it treats AI-assisted code migration as a powerful but fallible tool that requires robust human governance to be deployed ethically in high-stakes environments.

Several limitations of this work should be acknowledged. First, the framework I have derived is primarily from work experience at a single institution. While the ethical tensions identified are likely generalizable across regulated industries, the specific implementation of the framework may need to be adapted to the organizational structures, regulatory environments, and technical infrastructures of other institutions. Second, this paper does not present empirical data on the frequency or severity of errors in AI-assisted code migration, as such data is typically proprietary and not publicly available. Future research should seek to quantify these risks through controlled studies or anonymized industry surveys.

Third, the framework intentionally does not address the question of whether AI-assisted code migration should be used at all. The current position is that the technology is already in widespread use and that abstaining from it is not a realistic option for most institutions. The relevant ethical question is therefore not whether to use AI for code migration, but how to use it responsibly. The framework proposed here represents one answer to that question, grounded in the practical realities of enterprise software engineering.

It is also worth noting that the ethical framework presented in this paper is not static. As AI models improve in capability, as regulatory frameworks evolve to address AI-generated artifacts more explicitly, and as the industry develops better tooling for verification and provenance tracking, the specific practices recommended here will need to be updated. The principles, however—accountability, transparency, and evidence-based trust—are durable and should serve as the foundation for whatever specific practices emerge.

6. Conclusion

AI-assisted code migration represents a significant advancement in software engineering productivity, but its deployment in regulated financial systems introduces ethical tensions that existing frameworks have not adequately addressed. This paper has identified three such tensions—the accountability gap, the transparency deficit, and automation bias—and proposed a practitioner-grounded ethical framework consisting of mandatory human review checkpoints, provenance documentation, and differential testing protocols.

The central argument of this paper is that AI-driven efficiency gains in software development are not inherently value-neutral. Every decision to delegate code authorship to a machine learning system is simultaneously a decision about who bears responsibility for the output, who can verify its correctness, and how much trust is warranted in the absence of human review. In regulated industries, where the consequences of software failure are measured in financial harm, compliance violations, and erosion of public trust, these decisions carry substantial moral weight.

As organizations race to adopt AI-assisted development at scale, the frameworks governing that adoption must evolve at equal pace. This paper calls on software engineering leaders, compliance teams, and AI tool providers to co-develop standardized ethical review

processes for AI-generated code in regulated industries. The gap between AI capability and ethical accountability is widening. Closing it will require not only better tools, but a sustained commitment to the principle that the humans who deploy AI systems remain responsible for the outcomes those systems produce.

References

- ACM. (2018). ACM Code of Ethics and Professional Conduct. Association for Computing Machinery. <https://www.acm.org/code-of-ethics>
- Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 85–111.
- Coeckelbergh, M. (2022). *The Political Philosophy of AI: An Introduction*. Polity Press.
- Morley, J., Floridi, L., Kinsey, L., & Elhalal, A. (2020). From What to How: An Initial Review of Publicly Available AI Ethics Tools, Methods and Research to Translate Principles into Practices. *Science and Engineering Ethics*, 26, 2141–2168.
- O’Sullivan, S. (2024). Ethics of Software Programming with Generative AI: Is Programming without Generative AI Always Radical? *arXiv preprint*, arXiv:2408.10554.
- Parasuraman, R., & Manzey, D. H. (2010). Complacency and Bias in Human Use of Automation: An Attentional Integration. *Human Factors*, 52(3), 381–410.
- Sauvola, J., Tarkoma, S., Klemettinen, M., et al. (2024). Future of Software Development with Generative AI. *Automated Software Engineering*, 31, 26.
<https://doi.org/10.1007/s10515-024-00426-z>
- Techreviewer. (2025). AI in Software Development 2025: From Exploration to Accountability – A Global Survey Analysis. *Techreviewer Blog*.
<https://techreviewer.co/blog/ai-in-software-development-2025>
- Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. ACM.